Réseaux de neurones

Michaël Clément

michael.clement@enseirb-matmeca.fr

2025–26

Bordeaux INP – ENSEIRB-MATMECA – DE Expert Intelligence Artificielle

Acknowledgements:

- Aurélie Bugeau (*Univ. Bordeaux*)
- Charles Deledalle (*Brain Corp*)
- Hugo Larochelle (*Google Brain*)
- Andrew Ng (*Stanford*)
- Baptiste Pesquet (*ENSC*)

Objectives and organisation

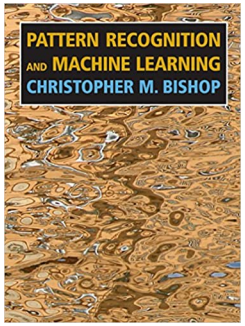
Objectives

- Understand how (deep) **neural networks** work
- Know the main **architectures** and **tools** in deep learning
- Be able to **implement** and **use** them in different **applications**

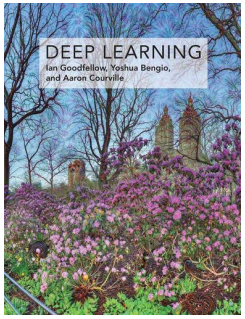
Organization

- **Two days of class**: mix of lectures and practices
- Information and class materials on Moodle
- **Grading**: final practice in May

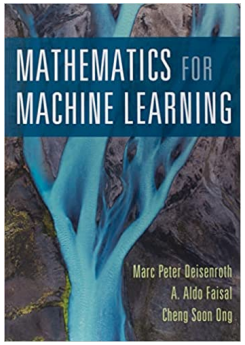
References



Pattern Recognition and Machine Learning
Christopher M. Bishop
`https://www.microsoft.com/en-us/research/people/cmbishop/prml-book/`



Deep Learning
Ian Goodfellow, Yoshua Bengio, Aaron Courville
`https://www.deeplearningbook.org`



Mathematics for Machine Learning
Marc Peter Deisenroth, A. Aldo Faisal, Cheng Soon Ong
`https://mml-book.com`

Schedule

Jour 1 :

- **Cours :** Introduction, Perceptron
- **TP1 :** Perceptron

Jour 2 :

- **Cours :** Multilayer perceptron (MLP)
- **TP2 :** MLP
- **Cours :** Convolutional neural networks (CNN), Deep learning
- **TP3 :** CNN

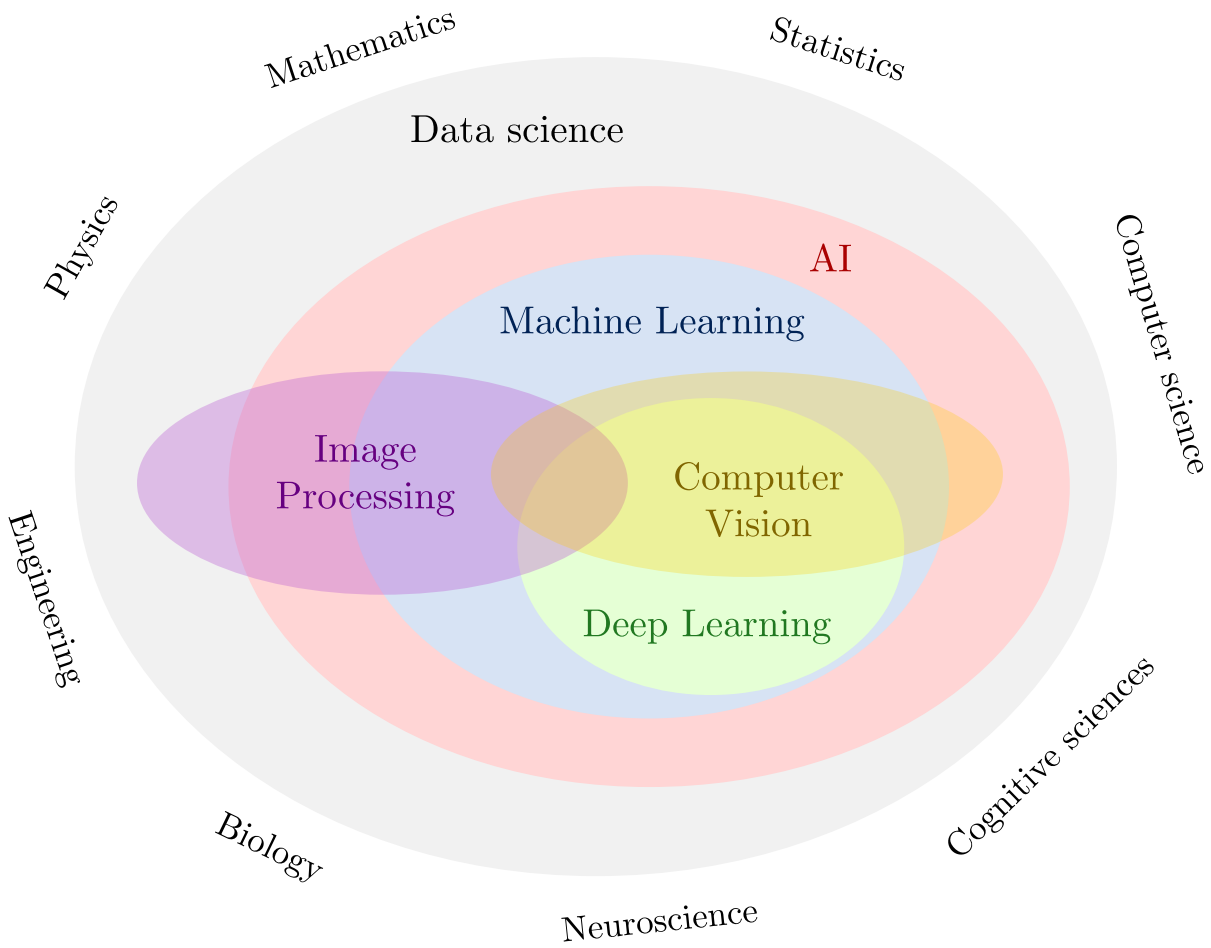
Jour 3: Projets, Module 10

En Mai: TP noté

Introduction

What is machine learning?

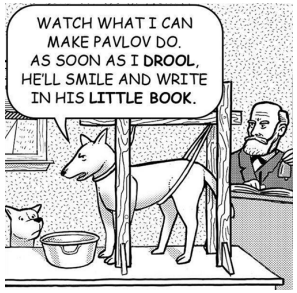
A multidisciplinary subject



Definitions

"Learning is any process by which a system improves performance from experience."

— ***Herbert Simon***



Pavlov's dog

(Mark Stivers, 2003)

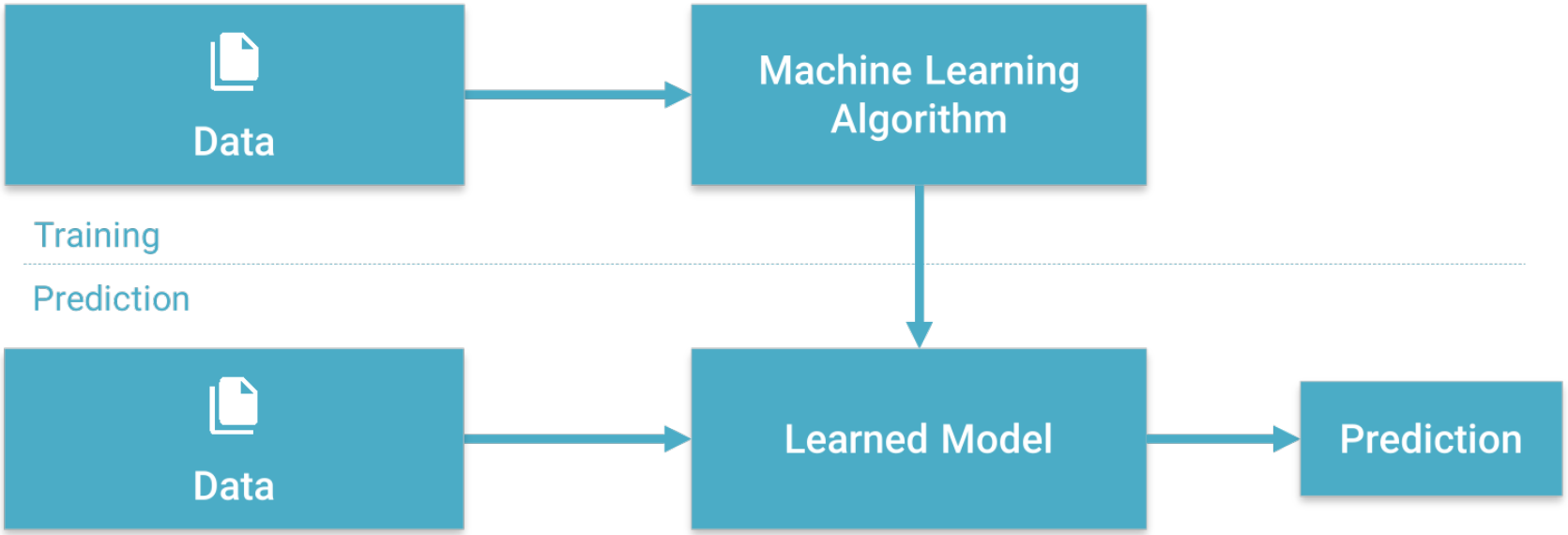
"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

— ***Tom Mitchell***

Learn from data

Machine learning provides various techniques that can learn from and make predictions on data.

Most of them follow the same general structure:



Main ingredients

To learn from examples, we will need:

1. Training data (examples):

$$\mathcal{D}_{\text{train}} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\},\ \boldsymbol{x}_i \in \mathbb{R}^d$$

2. Model (machine or program):

$$\underbrace{\boldsymbol{x}}_{\text{input data}} \quad \rightarrow \quad \underbrace{f(\boldsymbol{x}; \boldsymbol{w})}_{\text{function / algorithm}} \quad \rightarrow \quad \underbrace{\hat{\boldsymbol{y}}}_{\text{prediction}}$$

3. Loss, cost, objective function / energy:

$$\arg\min_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}; \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\})$$

Terminology

- **Sample (Observation or Data):** item to process (*e.g.*, classify).
  Example: an individual, a document, a picture, a sound, a video, etc.

Terminology

- **Sample (Observation or Data):** item to process (*e.g.*, classify).
  Example: an individual, a document, a picture, a sound, a video, etc.

- **Features (Input)**: set of distinct traits that can be used to describe each sample in a quantitative manner. Represented as a multi-dimensional vector usually denoted by $\boldsymbol{x}$.
  Example: size, weight, frequency, color, etc.

Terminology

- **Sample (Observation or Data):** item to process (*e.g.*, classify).
  Example: an individual, a document, a picture, a sound, a video, etc.

- **Features (Input)**: set of distinct traits that can be used to describe each sample in a quantitative manner. Represented as a multi-dimensional vector usually denoted by $\boldsymbol{x}$.
  Example: size, weight, frequency, color, etc.

- **Training set:** set of data used to discover predictive relationships.

Terminology

- **Sample (Observation or Data):** item to process (*e.g.*, classify).
  Example: an individual, a document, a picture, a sound, a video, etc.

- **Features (Input)**: set of distinct traits that can be used to describe each sample in a quantitative manner. Represented as a multi-dimensional vector usually denoted by $\boldsymbol{x}$.
  Example: size, weight, frequency, color, etc.

- **Training set:** set of data used to discover predictive relationships.

- **Testing set:** set of data used to assess the performance of a model.

Terminology

- **Sample (Observation or Data):** item to process (*e.g.*, classify).
  Example: an individual, a document, a picture, a sound, a video, etc.

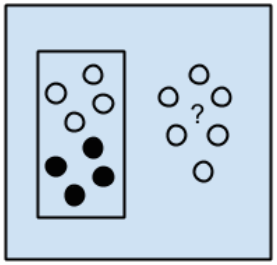
- **Features (Input)**: set of distinct traits that can be used to describe each sample in a quantitative manner. Represented as a multi-dimensional vector usually denoted by $\boldsymbol{x}$.
  Example: size, weight, frequency, color, etc.

- **Training set:** set of data used to discover predictive relationships.

- **Testing set:** set of data used to assess the performance of a model.

- **Label or Prediction (Output):** the class or outcome assigned to a sample. The actual prediction is often denoted by $\hat{\boldsymbol{y}}$ and the ground truth value by $\boldsymbol{y}$.
  Example: cat/dog, temperature, price, etc.

Learning approaches

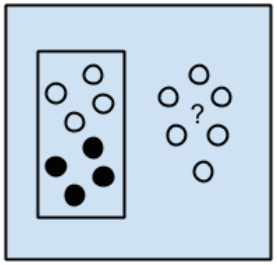


Supervised Learning Algorithms

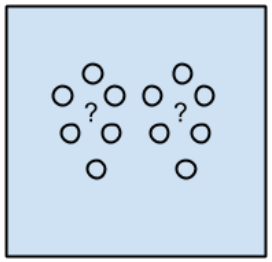
- **Supervised learning**: learn from labeled training data $\mathcal{D}_{\text{train}} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_N, \boldsymbol{y}_N)\}$

Types of learning

Learning approaches



Supervised Learning Algorithms

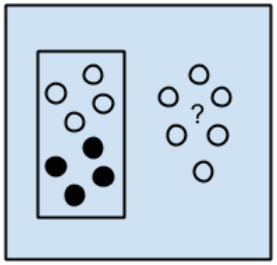


Unsupervised Learning Algorithms

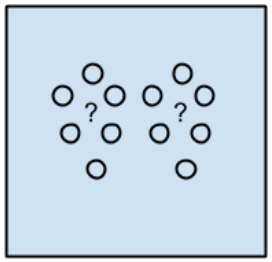
- **Supervised learning**: learn from labeled training data $\mathcal{D}_{\text{train}} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_N, \boldsymbol{y}_N)\}$

- **Unsupervised learning**: discover patterns in unlabeled training data $\mathcal{D}_{\text{train}} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$

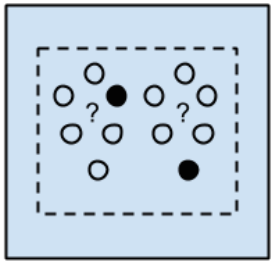
Learning approaches



Supervised Learning Algorithms



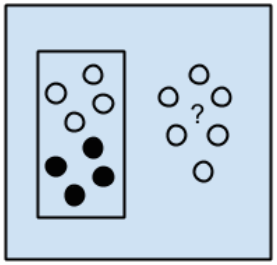
Unsupervised Learning Algorithms



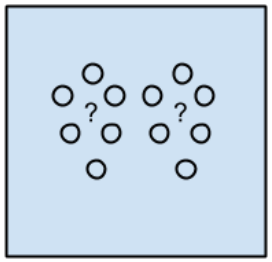
Semi-supervised Learning Algorithms

- **Supervised learning**: learn from labeled training data $\mathcal{D}_{\text{train}} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_N, \boldsymbol{y}_N)\}$
- **Unsupervised learning**: discover patterns in unlabeled training data $\mathcal{D}_{\text{train}} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$
- **Semi-supervised learning**: learn with a small amount of labeled data and a large amount of unlabeled data.

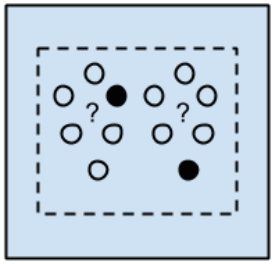
Learning approaches



Supervised Learning Algorithms



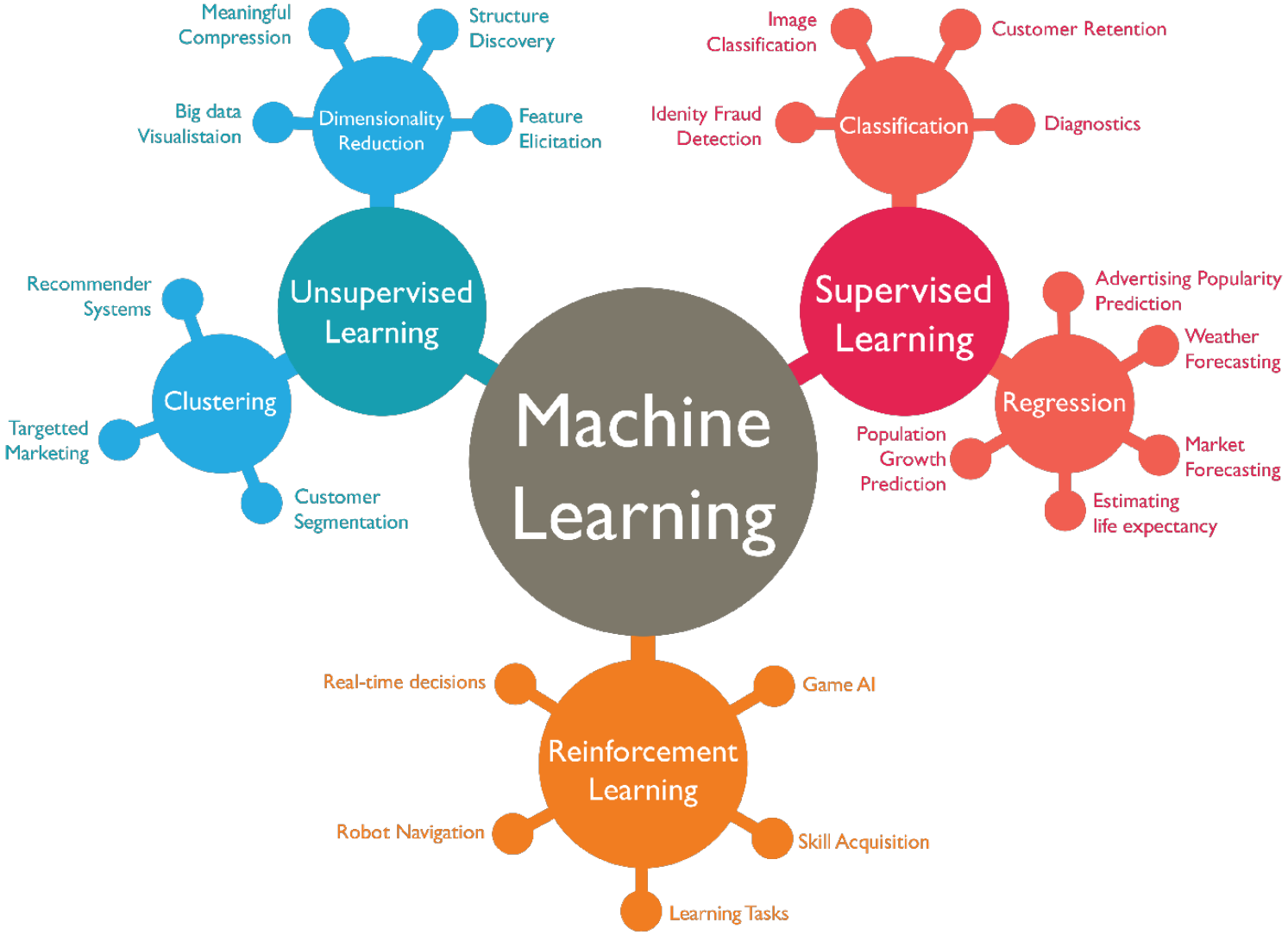
Unsupervised Learning Algorithms



Semi-supervised Learning Algorithms

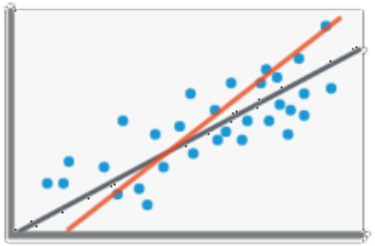
- **Supervised learning**: learn from labeled training data $\mathcal{D}_{\text{train}} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_N, \boldsymbol{y}_N)\}$
- **Unsupervised learning**: discover patterns in unlabeled training data $\mathcal{D}_{\text{train}} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$
- **Semi-supervised learning**: learn with a small amount of labeled data and a large amount of unlabeled data.
- **Reinforcement learning**: Learning based on feedback or reward.

Different tasks

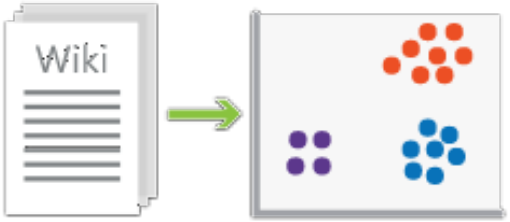


Problem types

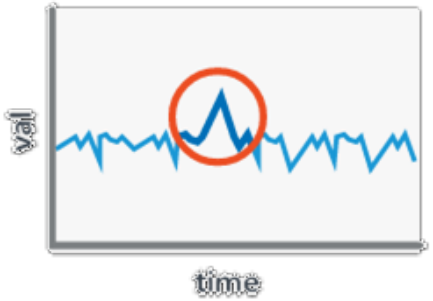
Classification
(supervised – predictive)



Regression
(supervised – predictive)



Clustering
(unsupervised – descriptive)



Anomaly Detection
(unsupervised– descriptive)

No free lunch theorem

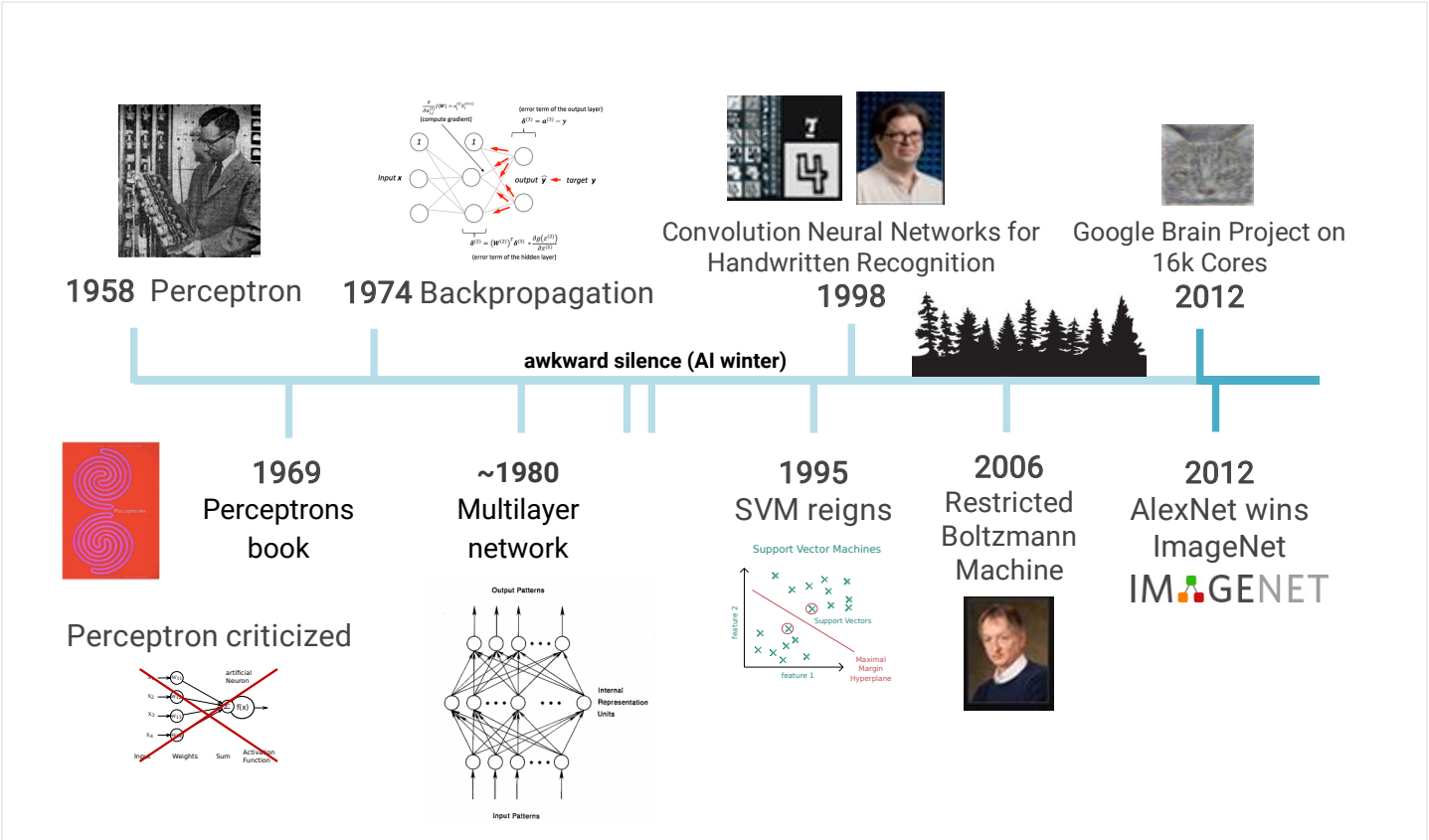
- Generic theorem in optimization theory and machine learning
- It states that, *averaged over all possible problems* and *without any prior knowlege*, **there is no objectively better learning algorithm**
- For example, a very fancy learning algorithm will, on average, perform as well as taking random decisions

No free lunch theorem

- Generic theorem in optimization theory and machine learning
- It states that, *averaged over all possible problems* and *without any prior knowlege*, **there is no objectively better learning algorithm**
- For example, a very fancy learning algorithm will, on average, perform as well as taking random decisions

$\rightarrow$ In practice, it means that we must design and apply solutions that are **biased towards the problem at hand**

Timeline of deep learning



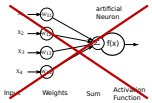
Perceptron

Perceptron

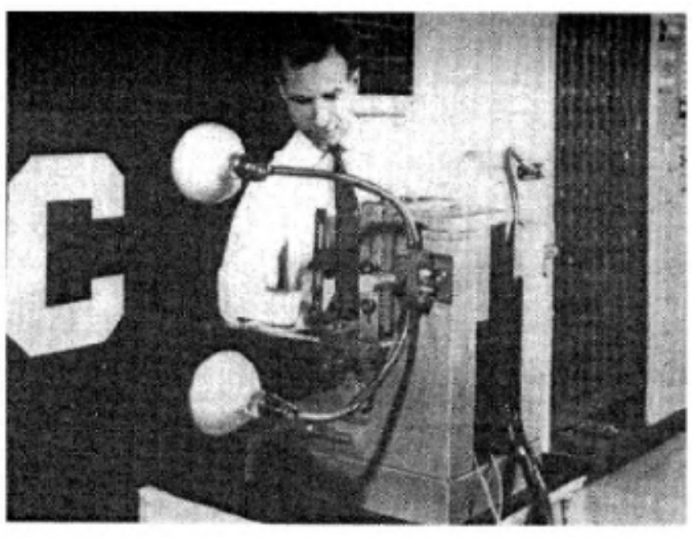
1958 Perceptron

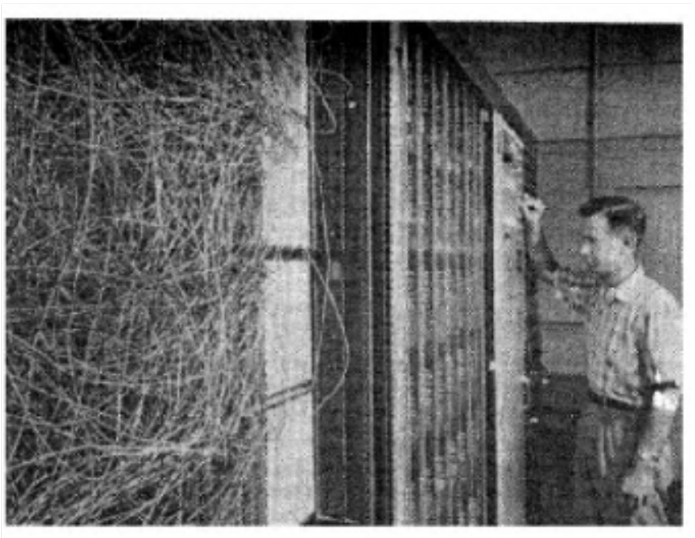
1969
Perceptrons
book

Perceptron criticized



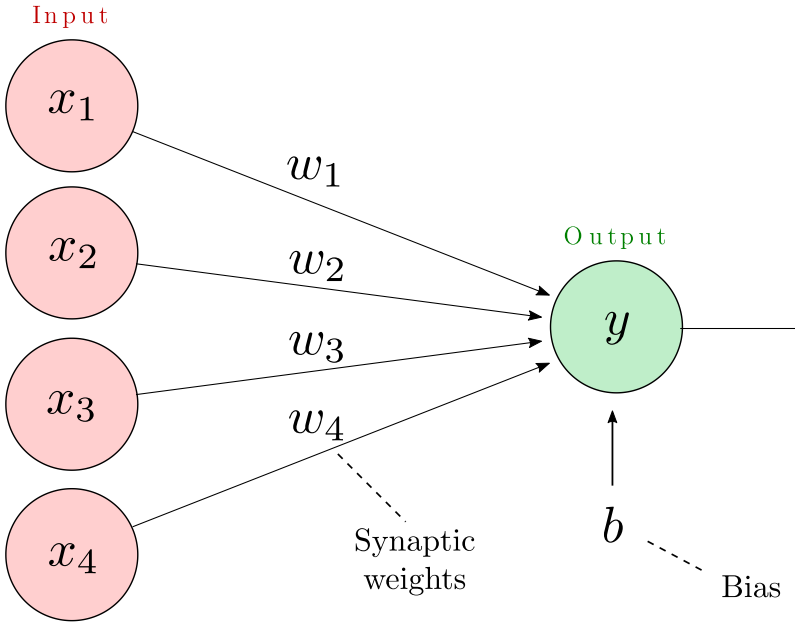
Perceptron (Frank Rosenblatt, 1958)





- First binary classifier based on supervised learning (discriminative)
- Foundation of modern artificial neural networks
- At that time: technological, scientific and philosophical challenges

Perceptron representation



$$\text{sign}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

$$y = \text{sign}\left(\sum_{i=1}^{4} w_i x_i + b\right)$$

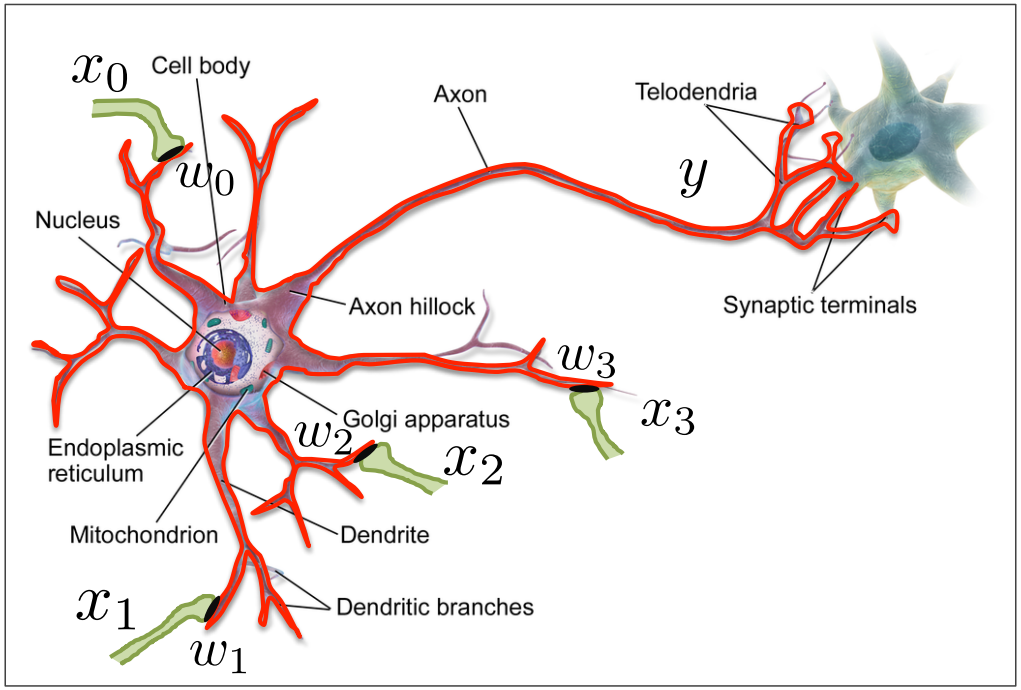
Parameters of the perceptron

- w_k: synaptic weights
- b: bias

$\Big\}$ $\longleftarrow$ real parameters to be estimated.

Training = adjusting the weights and biases

Biological inspiration



- When a neuron receives a stimulus with high enough voltage, it emits an **action potential** (also called a *spike*). It is said to **fire**.

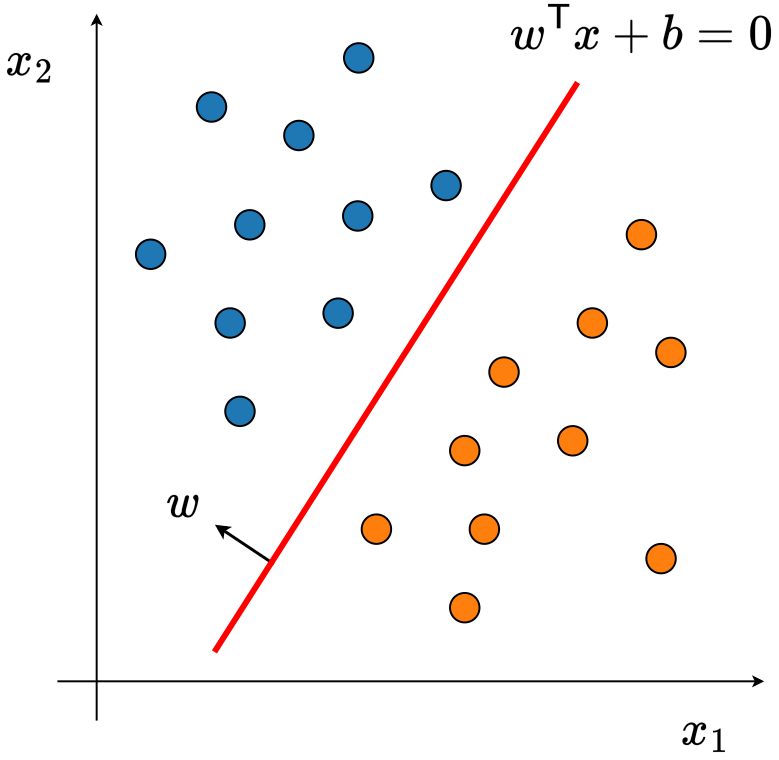
- The perceptron mimics this activation effect: it fires only when

$$\sum_i w_i x_i + b > 0$$

Geometrical point of view

The neuron models a **linear decision boundary**.

The parameters $\boldsymbol{w}$ and b define the equation of an hyperplane in the feature space:

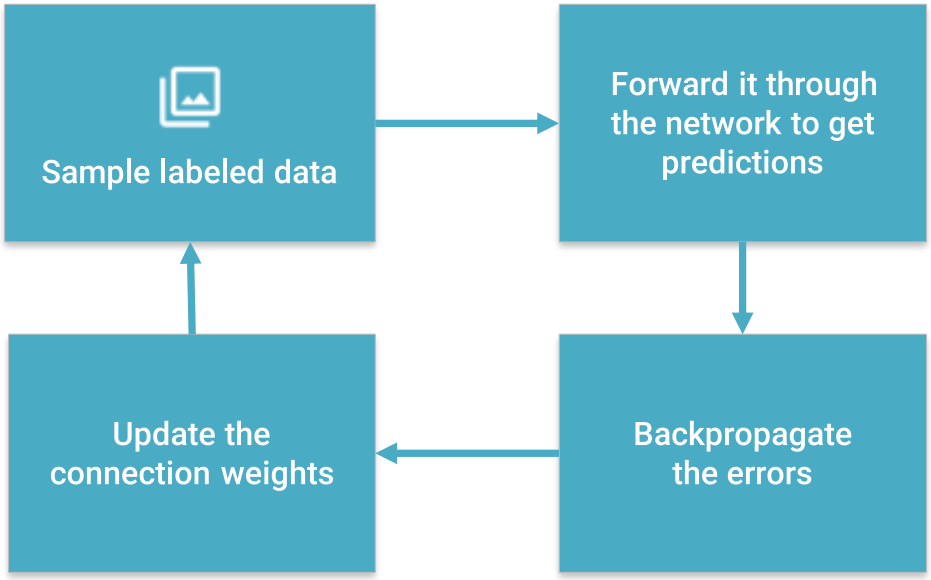


Training process

- To train a neural network over a large set of labeled data, we must continuously compute the difference between the network's predicted output and the ground truth labels.
- This difference is measured by a **loss function** $\mathcal{L}$, and the process for training a network is known as backpropagation, or backprop.
- During backprop, **weights and biases are updated** in order to minimize the loss.

$$\longrightarrow \textbf{Optimization}\text{: look for } \min_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w})$$

Training process



Learns by generating an error signal that measures the difference between the predictions of the network and the desired values and then using this error signal to change the weights (or parameters) so that predictions get more accurate.

Minimization of loss functions

Objective: $\min_{\boldsymbol{W}} \mathcal{L}(\boldsymbol{w})$

An example of **loss function** is the Mean Squared Error (MSE):

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

where the prediction of the model is:

$$\hat{y} = \sum_{j=1}^{K} w_j x_j + b = \boldsymbol{w}^\mathsf{T} \boldsymbol{x} + b$$

For this particular example, there exists a single global minimum, which can be found analytically. However, this is generally not the case. An efficient way to solve the problem is to use **gradient descent**.

Gradient descent

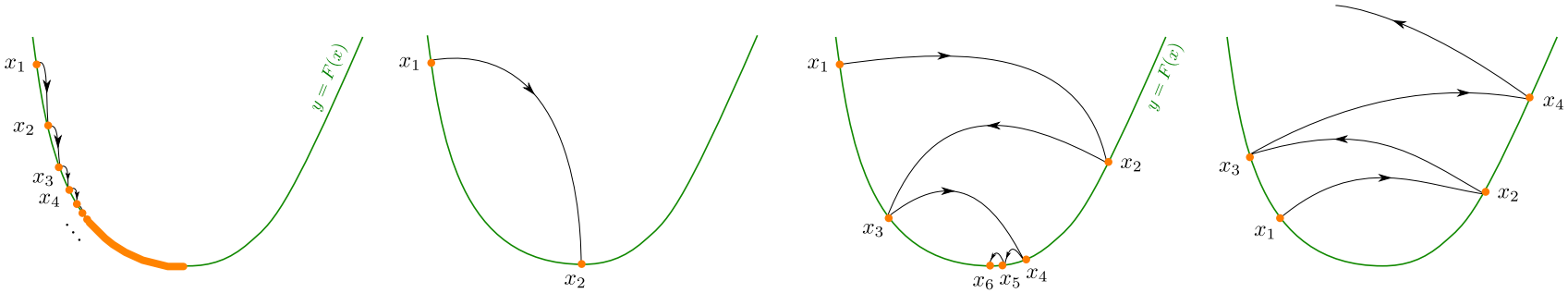
An iterative algorithm to find a minimum of a function

Principle

- Update the parameters by taking steps in the opposite direction of the gradient of the function to minimize:

$$\boldsymbol{w}^{(t+1)} \leftarrow \boldsymbol{w}^{(t)} - \gamma \nabla \mathcal{L}(\boldsymbol{w}^{(t)}), \ \gamma > 0.$$

- The hyperparameter γ is called the **learning rate** (or *step size*).



Small step size
Slow convergence

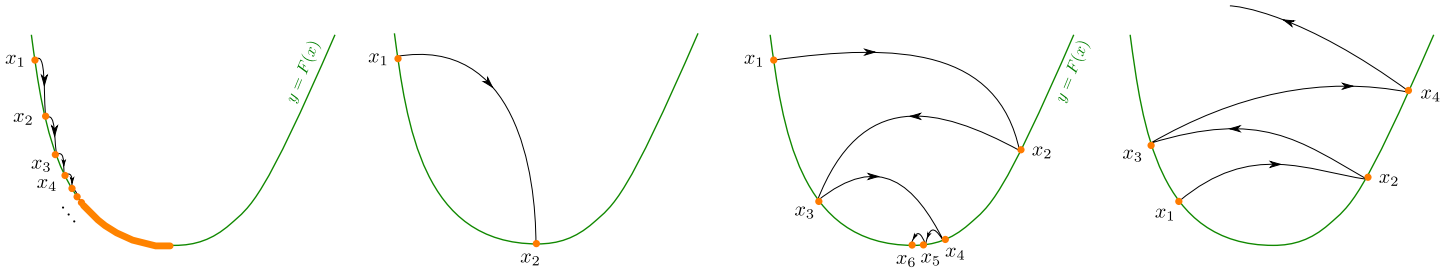
Good step size
Fast convergence

Large step size
Slow convergence

Too large step size
Divergence

Gradient descent – Examples

One dimension



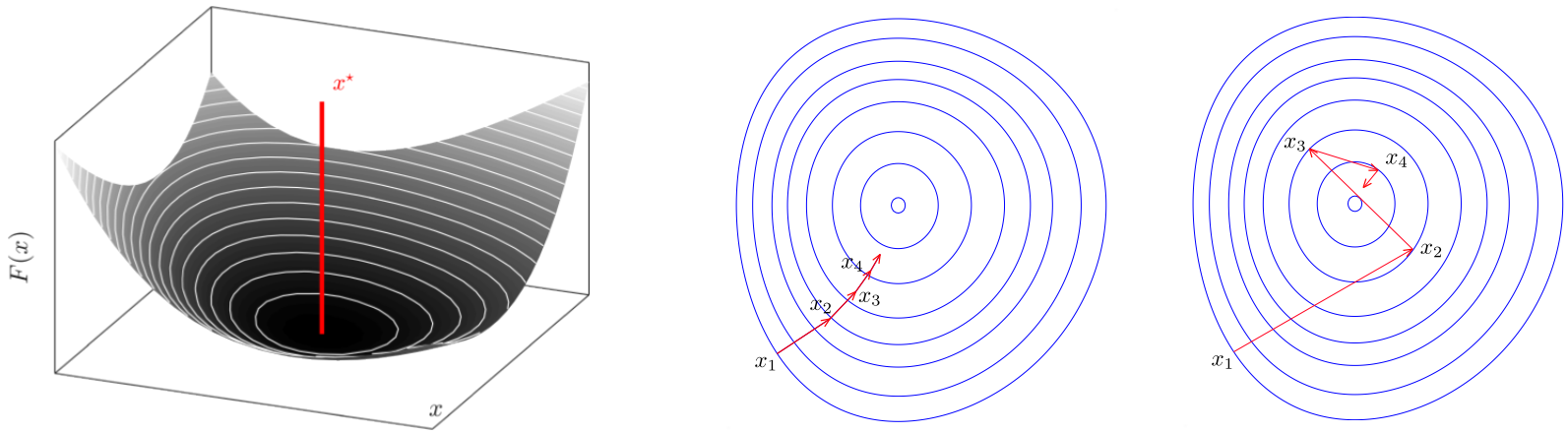
Small step size
Slow convergence

Good step size
Fast convergence

Large step size
Slow convergence

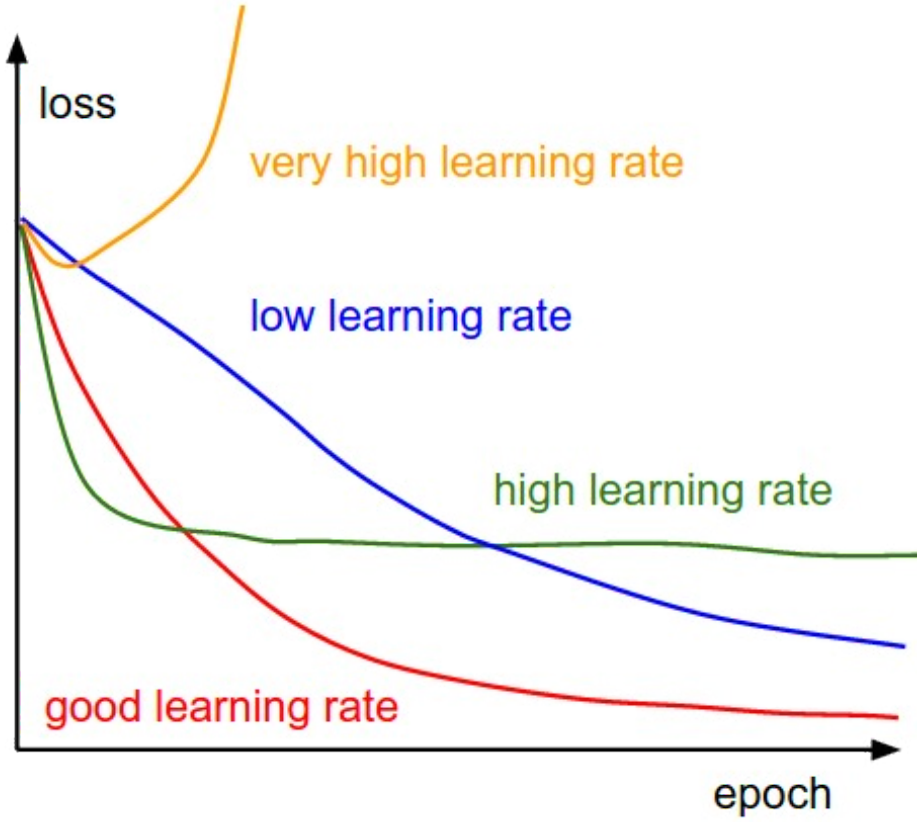
Too large step size
Divergence

Two dimensions



Gradient descent – Learning rate

Influence of the step parameter (learning rate)



The learning rate is an **hyperparameter** of the model.

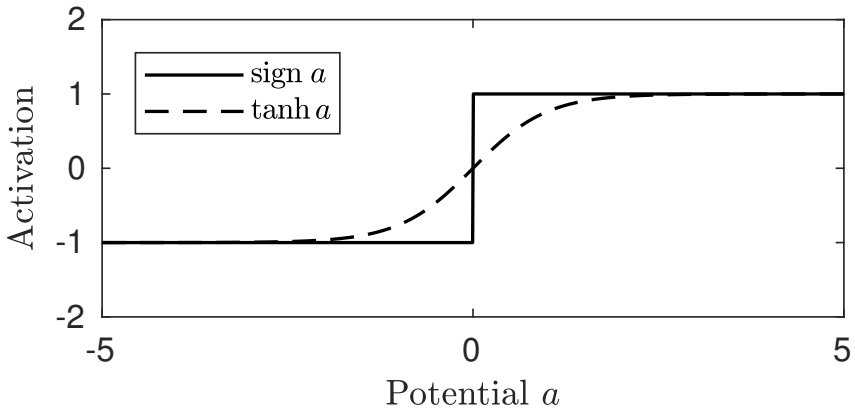
Activation functions

The neuron is usually paired with an activation function σ:

$$\hat{y} = \sigma\Big(\sum_{j=1}^{K} w_j x_j + b\Big) = \sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b)$$

The sign function is not differentiable, which is not ideal for gradient descent. In practice, it is more efficient to use differentiable activation functions such as $\tanh$ or $\mathrm{sigmoid}$:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in [-1, 1] \qquad \mathrm{sigmoid}(x) = \frac{1}{1 + e^{-x}} \in [0, 1]$$



Loss functions

Different loss functions for different problems

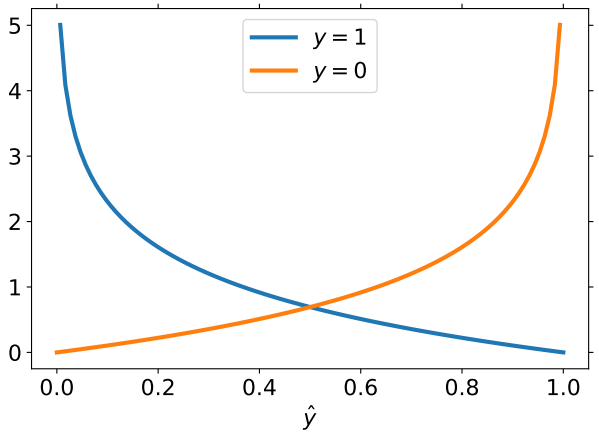
- **Regression**: mean squared error (MSE)

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- **Binary classification**: binary cross-entropy (BCE)

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)$$

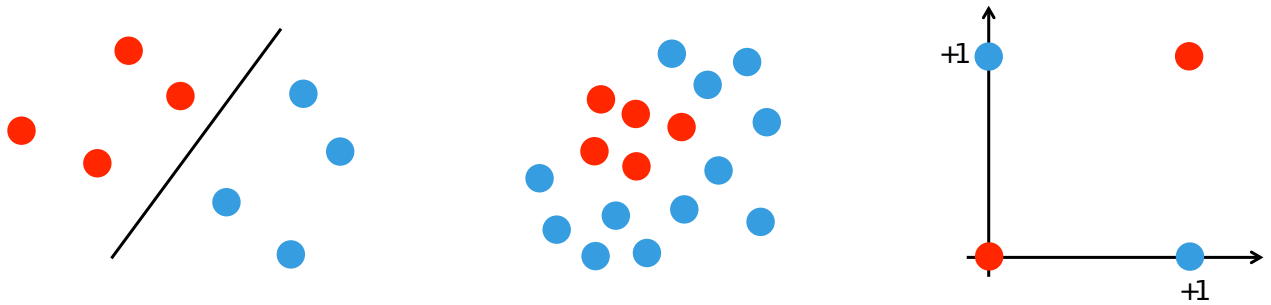
BCE should be used only with a $\mathrm{sigmoid}$ activation function.



Limitations of the percepron

Perceptrons book (Minsky and Papert, 1969)

A perceptron can only classify data points that are linearly separable:



Linearly separable Nonlinearly separable The xor function

Seen by many as a justification to stop research on perceptrons.

TP1 – Perceptron